

Azure Databricks Access Connector — When and Why It's Needed

Summary

An **Azure Databricks Access Connector** is a first-party Azure resource that holds a **managed identity** (system-assigned by default, or user-assigned). It is the mechanism Unity Catalog uses to authenticate to **Azure Data Lake Storage Gen2 (ADLS Gen2)** without storage account keys or service-principal secrets.

Bottom line: You do **not** need an access connector to create the Azure Databricks *workspace* itself. You **do** need one (in practice) to stand up a governed Unity Catalog environment — the metastore root storage, storage credentials, and external locations all rely on it. It is the standard, Databricks-recommended approach over access keys or service-principal secrets.

What the Access Connector Is

- A small, dedicated Azure resource (`Microsoft.Databricks/accessConnectors`) that connects a **managed identity** to your Azure Databricks account.
- By default it deploys with a **system-assigned managed identity**; you can instead attach a **user-assigned managed identity**.
- You grant its identity the **Storage Blob Data Contributor** role on the target ADLS Gen2 storage account, then reference the connector's **resource ID** when creating a storage credential or Unity Catalog metastore.
- The Azure user or service principal creating the connector must be a **Contributor or Owner** of an Azure resource group.

When You Need It vs. When You Don't

Scenario	Access Connector Required?
Deploy the Azure Databricks workspace (Portal / ARM / Bicep / Terraform)	No — the workspace, its managed resource group, and DBFS root are created automatically
Click around in a workspace for evaluation only	No
Create a Unity Catalog metastore with root storage	Yes (recommended pattern uses the connector's managed identity)

Create a storage credential for ADLS Gen2	Yes
Define external locations over ADLS Gen2 containers	Yes
Any production-grade, governed environment	Yes

Typical Setup Order

1. **Create the Azure Databricks workspace** (no connector needed yet).
2. **Create the Access Connector for Azure Databricks** in Azure.
3. **Grant the connector's managed identity** the *Storage Blob Data Contributor* role on the ADLS Gen2 storage account.
4. **Create a storage credential** of type *Azure Managed Identity*, supplying the connector's resource ID.
5. **Create external locations** that reference the storage credential.
6. **Create / attach the Unity Catalog metastore** using the connector for its root storage.

Why This Approach

Storage credentials store the managed identity, and external locations define a path to storage plus a reference to that credential. Using the access connector's managed identity means **no storage keys and no service-principal secrets** are kept anywhere — authentication is brokered through Azure's managed identity platform, which is more secure and is the Databricks-recommended pattern.

Microsoft Azure Reference URLs

[1] [Use Azure managed identities in Unity Catalog to access storage](#) — Primary guide: creating the access connector, choosing system- vs. user-assigned identity, and granting storage access.

[2] [Create a storage credential for connecting to Azure Data Lake Storage](#) — Step-by-step storage credential setup using the access connector's resource ID.

[3] [Connect to an Azure Data Lake Storage Gen2 \(ADLS Gen2\) external location](#) — Defining external locations over ADLS Gen2.

[4] [Create a Unity Catalog metastore](#) — Where the access connector is referenced for metastore root storage.

[5] [Connect to cloud object storage using Unity Catalog](#) — Overview of the storage-credential and external-location model.